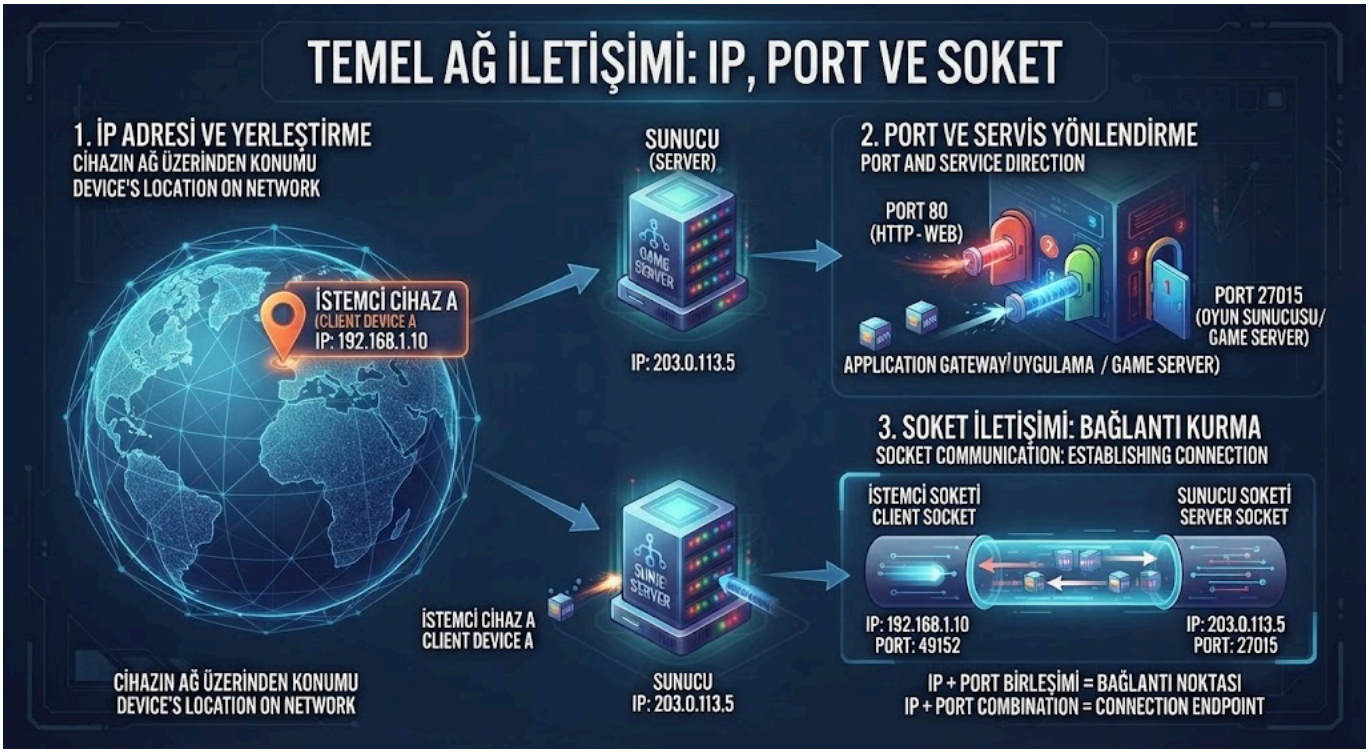
**Networking Temelleri - 1. **

Bu bölümde, çevrimiçi oyunların temelini oluşturan ağ iletişiminin yapı taşlarını inceleyeceğiz. Bir oyunun internetteki başka bir bilgisayarla nasıl konuştuğunu anlamak için IP, Port, Socket kavramlarını ve veri iletiminde kullanılan iki temel protokol olan TCP ile UDP arasındaki farkları detaylı olarak ele alacağız.

---

1. IP, Port ve Socket Kavramları

Bir oyun istemcisinin (sizin bilgisayarınız) bir oyun sunucusuna mesaj gönderebilmesi için, gönderilen verinin doğru adrese ulaşması gerekir. Bu adresleme mekanizması üç temel kavram üzerine kuruludur:



1.1. IP Adresi (Internet Protocol Address)

IP adresi, bir ağa bağlı her cihaza (bilgisayar, telefon, oyun konsolu, sunucu) atanan benzersiz sayısal etikettir. İnternet üzerindeki posta adresi gibi düşünülebilir.

- **İşlevi:** Veri paketlerinin doğru binaya (cihaza) ulaşmasını sağlar.
- **Türleri:**
  - **IPv4:** 32 bitlik adreslerdir (örnek: `192.168.1.1`). Günümüzde en yaygın kullanılan formattır, ancak adres sayısı sınırlıdır. (2^{32})
  - **IPv6:** 128 bitlik adreslerdir (örnek: `2001:0db8:85a3:0000:0000:8a2e:0370:7334`). Daha fazla adres imkanı sunar ve geleceğin standardıdır.
- **Oyunlardaki Önemi:** Bir oyun sunucusuna bağlanmak istediğinizde, DNS (Domain Name System) sunucusu, yazdığınız `oyun.com` gibi bir alan adını, sunucunun IP adresine çevirir. Ping değeriniz, büyük ölçüde sizin IP adresinizle sunucunun IP adresi arasındaki fiziksel uzaklığa ve yönlendirici (router) sayısına bağlıdır.

1.2. Port Numarası (Port Number)

Bir IP adresi, binanın adresidir. Ancak bir binada (bilgisayarda) aynı anda birden fazla uygulama çalışır. Örneğin, bir tarafta oyununuz açıkken, diğer tarafta bir web tarayıcısı veya Discord çalışıyor olabilir. Gelen veri paketinin hangi uygulamaya gitmesi gerektiğini belirlemek için port numarası kullanılır.

- **İşlevi:** IP adresine gelen veriyi, doğru uygulamaya (veya doğru oyun oturumuna) yönlendirir. Bir apartman dairesindeki daire numarası gibi düşünülebilir.
- **Aralığı:** 0 ile 65535 arasında değerler alır.
  - **0-1023:** Sistem portları (well-known ports). Örn: HTTP için 80, HTTPS için 443.
  - **1024-49151:** Kayıtlı portlar (registered ports). Örn: Çoğu oyun sunucusu bu aralıkta çalışır (Valve oyunları genelde 27015, Minecraft 25565).
  - **49152-65535:** Dinamik/Özel portlar (dynamic/private ports). İstemci uygulamalar tarafından geçici olarak kullanılır.
- **Oyunlardaki Önemi:** Bir oyun sunucusu kurarken, hangi portu dinleyeceğini (listen) belirtmeniz gerekir. İstemci oyuncular, bağlanmak için hem sunucunun IP adresini hem de bu port numarasını bilmelidir (örnek: `192.168.1.100:27015`).

1.3. Soket (Socket)

Soket, bir IP adresi ve bir port numarasının birleşiminden oluşan, iki uç nokta (endpoint) arasındaki iletişim kapısıdır.

- **Tanımı:** Soket = IP Adresi + Port Numarası. Veri göndermek veya almak için kullanılan programlama arayüzüdür (API).
- **İşlevi:** Bir istemci-sunucu iletişimini tanımlar. Örneğin, sunucu `213.74.23.100:8080` adresinde bir soket oluşturur ve bağlantıları bekler. İstemci, bu sokete bağlanarak veri alışverişine başlar.
- **Oyunlardaki Önemi:** Oyun motorları, arka planda soketler oluşturur. Her bağlantı, benzersiz bir soket çifti (Client Soket + Server Soket) ile temsil edilir. Bu sayede sunucu, binlerce bağlantıyı aynı anda yönetebilir.

---

2. TCP vs UDP Farkları (Oyun Perspektifi)

İki cihaz arasında veri göndermek için iki ana protokol kullanılır: TCP (Transmission Control Protocol) ve UDP (User Datagram Protocol). Bu iki protokol, oyun geliştirmede temel bir ayrım noktasıdır.

Özellik	TCP (Transmission Control Protocol)	UDP (User Datagram Protocol)
Bağlantı Türü	Bağlantı tabanlıdır (Connection-Oriented). Veri göndermeden önce 3 aşamalı bir el sıkışma (3-way handshake) ile bağlantı kurar.	Bağlantısızdır (Connectionless). Veriyi doğrudan gönderir, önceden bağlantı kurmaz.
Güvenilirlik (Reliability)	**Güvenilirdir.** Gönderilen her paketin karşı tarafa ulaşıp ulaşmadığını kontrol eder. Kayıp paketleri tekrar gönderir (Retransmission).	**Güvenilir değildir.** Paketlerin ulaşıp ulaşmadığını kontrol etmez. Kaybolan paketler için yeniden gönderme yapmaz.
Sıralama (Ordering)	Paketlerin sırasını garanti eder. Eğer paketler sırasız gelirse, doğru sıraya koyar.	Paketlerin sırasını garanti etmez. Sırasız gelebilirler.

Özellik	TCP (Transmission Control Protocol)	UDP (User Datagram Protocol)
Akış Kontrolü (Flow Control)	Veri akışını yönetir, alıcının kaldırabileceğinden fazla veri göndermesini engeller.	Akış kontrolü yoktur. Veriyi ne hızda göndereceği uygulamaya bağlıdır.
Hız (Speed)	Daha **yavaştır.** Güvenlik kontrolleri, paket sıralaması ve yeniden gönderme mekanizmaları ekstra zaman ve bant genişliği gerektirir.	Daha **hızlıdır.** Herhangi bir kontrol mekanizması olmadığı için "ateş et ve unut" (fire and forget) mantığıyla çalışır.
Kullanım Alanı (Oyun)	Kritik olmayan, sıralamanın ve ulaşmanın önemli olduğu veriler için.	Hızın kritik olduğu, paket kaybının tolere edilebildiği anlık veriler için.

TCP'nin "Hoş Geldin" Konuşması (3-Way Handshake):

1. İstemci: "Merhaba, bağlanmak istiyorum." (SYN)
2. Sunucu: "Tamam, hazırım." (SYN-ACK)
3. İstemci: "Anlaştık!" (ACK)

---

3. UDP vs TCP Kullanım Senaryoları Örnekleri

Hangi protokolün kullanılacağına oyunun türü ve gönderilecek verinin önemi karar verir. Günümüz oyunlarının çoğu, bu iki protokolü birlikte (hibrit) kullanır.

3.1. UDP'nin Kullanıldığı Senaryolar (Hız > Güvenilirlik)

- **Oyuncu Hareket Verileri (Position Updates):** Bir oyuncunun anlık konum ve yön bilgileri saniyede 10-60 kez gönderilir. Bu paketlerden birkaçı kaybolsa bile, bir sonraki gelen paketle durum güncellenir. Kaybolan eski bir hareket paketini yeniden göndermenin hiçbir anlamı yoktur, çünkü o oyuncu çoktan başka bir konuma gitmiştir. **UDP tercih edilir.**
- **Ateş Etme ve Mermi Verileri (Shooting / Projectiles):** Rekabetçi FPS oyunlarında (Valorant, CS:GO), atış anında gönderilen verilerin hızlı ulaşması hayati önem taşır. Arada bir paket kaybı yaşanırsa, "ben ateş ettim" hissi oyuncuda oluşmaz. Küçük kayıplar kabul edilebilir. **UDP tercih edilir.**
- **Sesli / Görüntülü İletişim (VoIP):** Oyun içi sesli sohbetlerde, birkaç milisaniyelik ses parçasının kaybolması, konuşmayı anlaşılmaz kılmaktan ziyade minik bir çatırtıya neden olur. Kayıp paketi tekrar göndermek, sesin gecikmeli gelmesine ve konuşmanın kopmasına yol açar. **UDP tercih edilir.**

3.2. TCP'nin Kullanıldığı Senaryolar (Güvenilirlik > Hız)

- **Oyun Girişi ve Kimlik Doğrulama (Login / Authentication):** Kullanıcı adı ve şifre bilgilerinin karşı tarafa eksiksiz ve hatasız ulaşması gerekir. Paket kaybı yaşanırsa şifre yanlış algılanabilir veya oturum açılamaz. **TCP tercih edilir.**
- **Karakter Envanteri ve Eşya Değişimleri (Inventory / Trading):** Bir oyuncunun envanterindeki eşyaların veya oyun içi satın alımların kaybolmaması veya çoğalmaması gerekir. Bu veritabanı işlemleridir ve sıralı, eksiksiz teslimat şarttır. **TCP tercih edilir.**

- **Oyunun Yama ve Güncelleme Dosyalarının İndirilmesi (Patches / Downloads):** Oyun dosyalarının (asset) doğru bir şekilde, bozulmadan inmesi gerekir. Hatalı bir bayt, oyunun çökmesine neden olabilir. **TCP tercih edilir (HTTP/HTTPS üzerinden).**

3.3. Hibrit Kullanım Örneği (Oyunlarda Genel Yaklaşım)

Modern 3A oyunlarda şöyle bir yaklaşım izlenir:

- **UDP Soketi:** Oyunun canlı oynanış verileri için (Konum, Ateş, Hasar, Animasyon tetikleyicileri) kullanılır. Buraya gelen veriler hızlı işlenir.
- **TCP Soketi (veya WebSocket):** Oyun içi sohbet mesajları, arkadaş listesi güncellemeleri, maç sonu istatistiklerin gönderilmesi gibi işlemler için kullanılır.

Özet: Hangi Oyun Hangi Protokolü Kullanır?

Oyun Türü	Tercih Edilen Protokol	Neden?
FPS (CS:GO, Valorant)	UDP (Oynanış) / TCP (Sunucu Bağlantısı)	Milisaniye hassasiyeti. Arada kaybolan bir atış verisi, eski veriyi beklemekten iyidir.
MOBA (LoL, Dota 2)	UDP (Oynanış) / TCP (Sohbet)	Anlık yetenek ve hareket verileri hızlı gitmeli. Sohbetin güvenli ulaşması gerekir.
MMORPG (WoW, FFXIV)	Hibrit (UDP ağırlıklı)	Dünyada dolaşma anlıktır (UDP), ancak envanter ve ticaret işlemleri sağlamdır (TCP).
Strateji (Age of Empires)	UDP (Genellikle)	Çok sayıda birimin anlık durum güncellemeleri hızlı iletilmelidir.
Dövüş (Street Fighter)	UDP (Neredeyse tümü)	Hata düzeltmesi yapmak yerine, bir sonraki frame'in gelmesi daha önemlidir.

Oyunlarda doğru protokolü seçmek, doğru aracı doğru iş için kullanmaya benzer. UDP, hız gerektiren, tolere edilebilir kayıpları olan oyun mantığı için tercih edilirken; TCP, veri bütünlüğünün ve sıralamanın mutlak öncelik olduğu sistemik işlemler için vazgeçilmezdir. Bir oyun geliştirirken, her veri türü için bu maliyet-fayda analizini yapmak, oyunun akıcılığını ve kararlılığını doğrudan belirler.

Basit TCP Chat Uygulaması

Bu bölümde, öğrendiğimiz TCP protokolü mantığını pekiştirmek için basit bir konsol tabanlı sohbet (chat) uygulaması geliştireceğiz. Bu uygulama, bir sunucu ve birden fazla istemcinin birbirleriyle mesajlaşabildiği temel bir sistem olacaktır.

Gerçek bir oyunun ağ altyapısı çok daha karmaşık olsa da, bu örnek, socket programlamanın, bağlantı yönetiminin ve mesaj iletiminin temel prensiplerini anlamak için mükemmel bir başlangıç noktasıdır.

---

1. Uygulama Mimarisi ve Akış Şeması

Mimari Yapı:

1. **TCP Sunucu (Server):** Bağlantıları kabul eder, gelen mesajları alır ve tüm bağlı istemcilere iletir (broadcast).
2. **TCP İstemci (Client):** Sunucuya bağlanır, mesaj gönderir ve sunucudan gelen diğer istemci mesajlarını ekrana yazdırır.

Veri Akışı:

1. Sunucu başlatılır ve belirli bir IP ve Port'u dinlemeye (listen) başlar.
2. Her istemci, sunucuya bir TCP bağlantısı kurar (3-way handshake).
3. İstemci, konsol üzerinden bir mesaj yazar ve "Enter" tuşuna basar.
4. İstemci, yazılan metni TCP soketi üzerinden sunucuya gönderir.
5. Sunucu, gelen mesajı alır ve *kendi mesajını gönderen istemci hariç* tüm bağlı istemcilere iletir.
6. Her istemci, sunucudan gelen mesajları sürekli dinleyen (listener) ayrı bir iş parçacığı (thread) ile anında ekrana yazdırır.

---

2. Kod Yapısı (Python ile Uygulama)

Bu uygulamayı, anlaşılırlığı ve kısa kod yapısı nedeniyle Python dilinde gerçekleştireceğiz. Python'un yerleşik `socket` ve `threading` kütüphanelerini kullanacağız.

2.1. TCP Sunucu Kodu (server.py)

Sunucu, sürekli çalışır, yeni istemcileri kabul eder ve her istemci için ayrı bir iş parçacığı oluşturarak mesajları yönetir.

```
import socket
import threading

# Sunucu ayarları
HOST = '127.0.0.1'  # Localhost (kendi bilgisayarınız)
PORT = 12345        # Dinlenecek port numarası

# Bağlı tüm istemcilerin socket'lerini tutacak liste
clients = []
nicknames = [] # İsteğe bağlı: Kullanıcı adları için

def broadcast(message, sender_client):
    """Gönderici hariç tüm bağlı istemcilere mesajı iletir."""
    for client in clients:
        if client != sender_client:
            try:
                client.send(message)
            except:
                # Hata durumunda istemciyi listeden kaldır
                remove_client(client)

def remove_client(client):
    """Bağlantısı kopan istemciyi listeden temizler."""
    if client in clients:
        index = clients.index(client)
        clients.remove(client)
```

```
        client.close()
        nickname = nicknames.pop(index) if index < len(nicknames) else
"Bilinmeyen"
        broadcast(f"{nickname} sohbetten ayrıldı.".encode('utf-8'), client)

def handle_client(client):
    """Her istemci için ayrı iş parçacığında çalışır, gelen mesajları
dinler."""
    while True:
        try:
            # İstemciden mesajı al (max 1024 byte)
            message = client.recv(1024)
            if not message:
                remove_client(client)
                break
            else:
                # Alınan mesajı diğer herkese ilet
                broadcast(message, client)
        except:
            remove_client(client)
            break

def start_server():
    """Sunucuyu başlatır ve gelen bağlantıları kabul eder."""
    server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    server.bind((HOST, PORT))
    server.listen()
    print(f"Sunucu {HOST}:{PORT} adresinde çalışıyor...")

    while True:
        # Yeni bağlantıyı kabul et
        client, address = server.accept()
        print(f"Yeni bağlantı: {address}")

        # İstemciden kullanıcı adını al
        client.send("KULLANICI_ADI".encode('utf-8'))
        nickname = client.recv(1024).decode('utf-8')
        nicknames.append(nickname)
        clients.append(client)

        # Katılım mesajını herkese duyur
        broadcast(f"{nickname} sohbete katıldı!".encode('utf-8'), client)
        client.send("Sunucuya bağlandınız!".encode('utf-8'))

        # Bu istemci için yeni bir iş parçacığı başlat
        thread = threading.Thread(target=handle_client, args=(client,))
        thread.daemon = True # Sunucu kapanınca thread de kapansın
        thread.start()

if __name__ == "__main__":
    start_server()
```

2.2. TCP İstemci Kodu (client.py)

İstemci, sunucuya bağlanır, kullanıcı adını gönderir ve ardından hem kullanıcı girdisini (mesaj gönderme) hem de sunucudan gelen mesajları (mesaj alma) aynı anda yönetmek için iki iş parçacığı kullanır.

```
import socket
import threading

# Sunucu bilgileri
HOST = '127.0.0.1'
PORT = 12345

def receive_messages(client):
    """Sunucudan gelen mesajları sürekli dinler ve ekrana yazdırır."""
    while True:
        try:
            message = client.recv(1024).decode('utf-8')
            if message == "KULLANICI_ADI":
                # Sunucu kullanıcı adı istiyor
                nickname = input("Kullanıcı adınızı girin: ")
                client.send(nickname.encode('utf-8'))
            else:
                # Gelen mesajı ekrana yaz
                print(message)
        except:
            # Bağlantı koptuysa çık
            print("Sunucu ile bağlantı kopmuş!")
            client.close()
            break

def start_client():
    """İstemciyi başlatır, sunucuya bağlanır ve mesaj gönderme/alma
işlemlerini yönetir."""
    client = socket.socket(socket.AF_INET, socket.SOCK_STREAM)

    try:
        client.connect((HOST, PORT))
    except:
        print("Sunucuya bağlanılamadı!")
        return

    # Mesaj alma işlemini ayrı thread'de başlat
    receive_thread = threading.Thread(target=receive_messages, args=
(client,))
    receive_thread.daemon = True
    receive_thread.start()

    # Ana thread'de mesaj gönderme işlemi
    while True:
        message = input()
        if message.lower() == '/quit':
            client.close()
            break
        try:
```

```
            client.send(message.encode('utf-8'))
        except:
            print("Mesaj gönderilemedi!")
            break

if __name__ == "__main__":
    start_client()
```

---

3. Kodun Adım Adım Çalışma Mantığı

1. **Sunucu Başlatılır:** `server.py` çalıştırılır. Sunucu `127.0.0.1:12345` adresinde beklemeye başlar.
2. **İstemci Bağlanır:** `client.py` çalıştırılır. İstemci sunucuya TCP bağlantısı kurar.
3. **Kullanıcı Adı Gönderilir:** Sunucu, istemciye özel bir "KULLANICI_ADI" mesajı gönderir. İstemci bu mesajı alınca konsoldan isim girmesini bekler ve gönderir.
4. **Mesaj Gönderme:** İstemci konsoluna bir metin yazılır ve Enter'a basılır. Bu metin, TCP soketi üzerinden sunucuya gönderilir.
5. **Sunucu İşlemi:** Sunucu, `handle_client` fonksiyonunda bu mesajı alır ve `broadcast` fonksiyonunu çağırarak *gönderen istemci hariç* tüm bağlı istemcilere iletir.
6. **Mesaj Alma:** Diğer istemciler, `receive_messages` thread'inde sürekli bekleyen `client.recv()` fonksiyonu sayesinde mesajı anında alır ve ekrana yazdırır.

---

4. TCP Chat Uygulamasından Çıkarımlar (Oyun Geliştirme Perspektifi)

Bu basit uygulama, daha büyük oyun altyapılarının temelini oluşturan birkaç kritik prensibi gösterir:

Prensip	Chat Uygulamasındaki Karşılığı	Oyundaki Karşılığı
Server Authority	Sunucu, tüm mesajları toplar ve dağıtır. Hiçbir istemci diğerine doğrudan mesaj göndermez.	Sunucu, tüm oyuncu hareketlerini ve atışlarını doğrular. İstemciler doğrudan birbirlerine "vurdum" diyemez.
Broadcasting (Yayın)	Bir istemcinin mesajı, diğer tüm istemcilere iletilir.	Bir oyuncunun pozisyonu, diğer tüm oyuncuların ekranında güncellenir.
Threading (Çoklu İş Parçacığı)	Her istemci için ayrı bir thread, mesajları bekler.	Sunucu, her oyuncu için ayrı bir "işlem" yürütür. Gelen veriler sırayla işlenir.
Connection Handling	İstemci bağlantısı koptuğunda (`/quit`), diğerlerine duyurulur ve temizlenir.	Bir oyuncunun oyunu kapattığında veya bağlantısı koptuğunda, oyundan çıkarılır ve diğer oyunculara bildirilir.
Message Protocol	Mesajlar düz metin olarak gönderilir. "KULLANICI_ADI" özel bir komuttur.	Gerçek oyunlarda, veriler JSON veya binary (ikili) formatlarda sıkıştırılarak gönderilir. Paket türleri (MOVE, SHOOT, STATE_UPDATE) bu şekilde ayrıştırılır.

---

5. İleri Çalışma Önerileri

Bu uygulamayı daha da geliştirerek oyun ağ yapısına yaklaştırabilirsiniz:

1. **Özel Mesaj (Private Message):** `msg Ahmet Merhaba` komutu ile sadece belirli bir kullanıcıya mesaj gönderecek mantığı ekleyin. (Bu, oyunlardaki "hedefleme" ve "özel durum bildirimi" gibi işlemlere benzer.)
2. **Zaman Damgası (Timestamp):** Her mesaja gönderim anının zamanını ekleyin. (Oyunlardaki paket sıralaması ve gecikme hesaplamalarına temel oluşturur.)
3. **JSON Formatı:** Mesajları düz metin yerine JSON formatında gönderin. (Örn: `{"type":"chat", "sender":"Ali", "content":"Merhaba"}`). Bu, kodun daha yapılandırılabilir olmasını sağlar.
4. **Bağlantı Kopma Yönetimi:** Sunucunun, kalp atışı (heartbeat) mekanizması ile istemcilerin hala bağlı olup olmadığını periyodik olarak kontrol etmesini sağlayın.

Bu TCP Chat uygulaması, istemci-sunucu mimarisinin en temel ve sağlam örneğidir. TCP'nin güvenilir, sıralı veri iletimi özelliği, bu tür sohbet uygulamaları için idealdir. Oyuna geçtiğimizde, UDP'nin hız avantajı devreye girse de, bağlantı yönetimi, thread kullanımı ve mesaj iletim mantığı (broadcast) buradakiyle neredeyse aynıdır. Bu kodu anlamak, oyunlardaki "Netcode" (Ağ Kodu) mantığının %70'ini anlamak anlamına gelir.